

Lab2 实验报告

23373428

马祺言

一、思考题

Thinking 2.1

均为虚拟地址。

Thinking 2.2

使用宏定义后，可以保证在实现不同类型的链表的时候，不需要重写通用功能的代码，提高了可重用性。

性能差异：

插入均为 $O(1)$ ，删除单向链表为 $O(n)$ ，其余为 $O(1)$

Thinking 2.3

C

```
#define LIST_HEAD(name, type)
```

```
    struct name {
```

```
        struct type *lh_first; /* first element */
```

```
    }
```

```
#define LIST_ENTRY(type)
```

```
    struct {
```

```
        struct type *le_next; /* next element */
```

```
        struct type **le_prev; /* address of previous next element
```

```
    */
```

```
};
```

Thinking 2.4

不同进程通过 TLB 将同一个程序地址空间映射到不同的物理空间，从而使用独立的地址空间，有了 ASID 可以在访问 TLB 的时候通过对比 ASID 来确认是否命中了正确的进程对应数据，而不会导致访问其他进程的数据的物理地址。

Figure 3-7 show the contents of one of the 16 dual-entries in the JTLB.

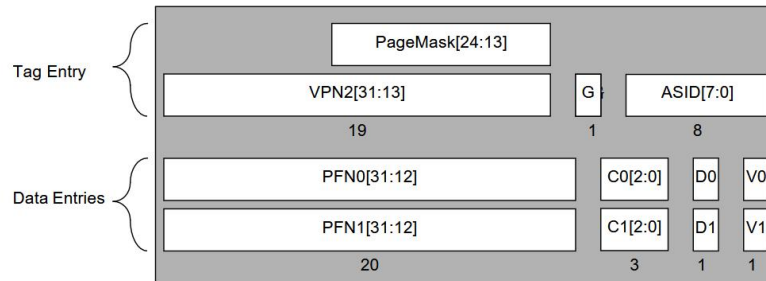


Figure 3-7 JTLB Entry (Tag and Data)

ASID 共八位，可容纳 256 地址空间

Thinking 2.5

```
12 */
13 void tlb_invalidate(u_int asid, u_long va) {
14     tlb_out((va & ~GENMASK(PGSHIFT, 0)) | (asid & (NASID - 1)));
15 }
16 /* End of Key Code "tlb_invalidate" */
```

tlb_invalidate 调用 tlb_out;

删除特定虚拟地址在 TLB 中的旧表项;

```
1
2
3 LEAF(tlb_out)
4 .set noreorder
5     mfc0    t0, CP0_ENTRYHI #把ENTRYHI的数据存储到t0
6     mtc0    a0, CP0_ENTRYHI #把目标页表项的key存储到ENTRYHI
7     nop
8     /* Step 1: Use 'tlbp' to probe TLB entry */
9     /* Exercise 2.8: Your code here. (1/2) */
10    tlbp #在tlb中查找对应页表项并把索引存到index
11    nop
12    /* Step 2: Fetch the probe result from CP0.Index */
13    mfc0     t1, CP0_INDEX #把索引从index存到t1
14    .set reorder
15    bltz     t1, NO_SUCH_ENTRY #如果小于0则没找到跳转到NO_SUCH_ENTRY,
16    .set noreorder #大于等于0, 找到了
17    mtc0     zero, CP0_ENTRYHI #给ENTRYHI置零
18    mtc0     zero, CP0_ENTRYLO0 #给ENTRYLO0置零
19    mtc0     zero, CP0_ENTRYLO1 #给ENTRYLO1置零
20    nop
21    /* Step 3: Use 'tlbwi' to write CP0.EntryHi/Lo into TLB at CP0.Ind
22    ex */
23    /* Exercise 2.8: Your code here. (2/2) */
24    tlbwi #把ENTRYHI、ENTRYLO0、ENTRYLO1中的值写入索引指定的表项
25    .set reorder
26    NO_SUCH_ENTRY:
27        mtc0    t0, CP0_ENTRYHI #没找到则写回ENTRYHI的值
28        j       ra #回到调用函数处
29    END(tlb_out)
30
```

Thinking 2.6

CPU 尝试访问数据，如果 TLB 缺失：

1. 调用 `do_tlb_refill`
2. 调用 `_do_tlb_refill`，进行 TLB 重填
3. 调用 `page_lookup`，查找给定虚拟地址对应页表项
4. 找不到对应页表项，调用 `passive_alloc` 进行被动页面分配
5. 调用 `page_alloc`，创建页表
6. 调用 `page_insert`，将虚拟地址映射到该物理页面
7. 调用 `tlb_invalidate`，删除特定虚拟地址的映射，保证下次访问相应虚拟地址时触发 tlb 重填

Thinking 2.7

X86 架构使用分段、分页机制来管理内存，将逻辑地址转化为线性地址，再通过页表转化为逻辑地址，仅需要 MMU 硬件管理 TLB。

mips 不使用分段机制而只使用分页机制，通过虚拟地址到物理地址的转换来进行访存，同时依靠软件与硬件来管理 TLB。

二、实验难点

本次实验题量较大，相较于前两次实验，最大的难点是涉及了较多对于内存管理的知识，各个文件、函数之间的相互调用、顺序逻辑相当复杂，幸而在逐个击破后还有整理回顾环节，得以加深理解。

此外，这次实验设计了部分较难理解的部分，比如双向链表中，`**le_prev` 是一个指向该节点前一个节点的 `*le_next` 的指针，即指向前一个节点指向当前节点的指针，十分费解，在插入节点的时候捣大乱。

三、体会

好难 T_T

通过这次实验，我对内存管理有了更深的理解，同时了解了宏、mips 与 C 语言相互调用，以及双向链表等知识。